

Guía personal de estudio y defensa de CAT-IA

Autoras: Antonella Cuvertino y Miriam Hammami.

Lo primero que debes poder decir

«CAT-IA ayuda a una mesa de soporte a clasificar tickets y preparar respuestas con evidencia. El caso es Nexo TI, una empresa ficticia. El sistema recupera políticas internas simuladas y guías externas reales de Mozilla, construye contexto para un LLM, valida su salida y muestra un borrador a un operador. El operador conserva la decisión final».

Aprende esta idea con tus palabras. No memorices resultados como si fueran universales: muestra el informe que produjo tu propia ejecución. La rúbrica también evalúa resolver un cambio sin apoyo externo; debes practicar personalmente y poder trabajar sin esta guía durante ese segmento.

Diferencia con tu propuesta original

Antes: ticket → prompt → LLM → texto JSON. Ahora: validación → recuperación interna/externa → contexto → LLM → validación de contrato/citas → borrador con traza y revisión. Además, prioridad por política explícita, manejo de ausencia de datos, pruebas y métricas separadas. El aporte de RAG se encuentra antes de generar, no consiste simplemente en guardar documentos junto al código.

Conceptos que debes dominar

LLM: modelo que genera texto a partir de patrones aprendidos. No es una base de datos de la empresa y puede producir afirmaciones plausibles falsas. Prompt: instrucciones y contexto enviados al modelo.

Tokens: unidades con las que procesa texto, distintas de palabras y caracteres. Ventana de contexto: capacidad limitada de entrada y generación.

Zero-shot: dar instrucciones sin ejemplos. Few-shot: incluir pocos ejemplos de criterio. En v3 hay ejemplos para distinguir contratación de soporte, reportes de seguridad y sugerencias. La prioridad se calcula fuera del LLM. No se pide razonamiento interno paso a paso, sino una justificación verificable.

RAG: recuperar, aumentar contexto y generar. Ingesta: preparar fuentes para poder buscarlas. Chunk: fragmento de documento. Overlap: palabras compartidas entre fragmentos vecinos. TF-IDF: pondera términos por frecuencia en documento y rareza en corpus; produce vectores dispersos. Embedding denso: representación aprendida que puede aproximar relaciones semánticas. Coseno: compara dirección de vectores. Top-k: límite de fragmentos recuperados. Umbral: mínimo puntaje admitido. Ninguno es porcentaje de confianza del LLM.

Alucinación: contenido no respaldado o incorrecto. Abstención: reconocer falta de evidencia y derivar.

Ground truth: etiqueta de referencia revisable; no es verdad incuestionable por estar en un JSON.

Faithfulness: respaldo de afirmaciones por el contexto. Relevancia: qué tan bien responde a la necesidad.

Precisión de recuperación: proporción de recuperado que sirve. Recall: proporción de lo relevante que se recuperó.

Agente acotado: coordina un conjunto definido de pasos y herramientas sin modificar sistemas externos. CAT-IA es un workflow de asistencia, no un agente autónomo de propósito general que elige cualquier herramienta mediante function calling. Explicar esta frontera muestra dominio del diseño.

Cómo seguir el código en cinco minutos

Abre `catia/api.py`. La ruta POST recibe un Ticket validado y llama `Agent.run`. Abre `catia/schemas.py`: aquí están longitudes, categorías y JSON esperado. Luego `catia/agent.py`: `redact`, `search`, `priority_for`, `generate`, `validate_evidence` y respuesta con traza. Este archivo es el recorrido principal de la demo.

Abre `catia/retrieval.py`. `load_chunks` lee el manifiesto, verifica hash y corta ventanas. Retriever crea la matriz TF-IDF. `search` vectoriza la consulta, calcula similitud, filtra, ordena y limita contexto. Si hay embeddings, calcula también la señal densa. Abre `catia/generation.py`: diferencia `DemoGenerator` de `OllamaGenerator` y muestra `build_messages`.

Abre `prompts/v3.txt` y señala rol, restricciones, ejemplos y formato. Abre `config/rag.json` y relaciona cada número con el comportamiento. Abre `data/sources.json` y localiza una interna y una externa. Finalmente abre `tests/test_system.py` y `scripts/evaluate.py`: explica qué comprueba una prueba y qué mide una evaluación.

Guion sugerido de exposición de 12 minutos

0:00–1:00: problema, organización ficticia y usuario. 1:00–2:00: objetivos y alcance. 2:00–3:30: fuentes y diagrama. 3:30–4:30: prompts y control de contexto. 4:30–7:30: demo con caso Firefox y caso sin evidencia. 7:30–9:00: pruebas y métricas, identificando modo ejecutado. 9:00–10:00: decisiones y límites. 10:00–12:00: cambio pequeño y conclusión sobre próximos pasos. Ajusta al tiempo asignado por el docente; este reparto no sustituye su instrucción.

Integrante A puede cubrir problema, datos y recuperación. Integrante B puede cubrir integración, validación y pruebas. Ambos deben poder intercambiar papeles y contestar cualquier pregunta.

Demo paso a paso y explicación

Abre la aplicación y di qué modo muestra. Si dice simulación, no digas que está respondiendo un LLM. Envía: «El portal no inicia sesión en Firefox. ¿Cómo reviso cookies y datos del sitio?». Explica qué es un ticket, de dónde salen los documentos y qué diferencia hay entre política y guía externa. Muestra el puntaje y aclara que es similitud.

Lee el borrador y encuentra una cita en el fragmento. Explica que comprobar una copia exacta ayuda a detectar referencias inventadas, pero que tú verificas si esa evidencia realmente sostiene la recomendación. Muestra la traza: id, prompt y corpus. Después prueba «Distancia entre Júpiter y Neptuno». Describe por qué abstenerse es mejor que inventar una respuesta empresarial.

Prueba «URGENTE quiero conocer sus planes». Mantén impacto individual y sin caída. Prioridad esperada Baja. Explica que las categorías las propone el generador y que la prioridad proviene de una regla; esto reduce variación para una decisión simple y contractual. Las horas mostradas son de primera respuesta, no de reparación garantizada.

Cambios en vivo para practicar sin ayuda

Ejercicio 1: el docente pide reducir ruido. Cambia top-k de 4 a 2 en la pantalla, repite el mismo ticket y observa qué evidencia se pierde. Si necesitas medir, ejecuta `scripts.evaluate --split dev --top-k 2` y compara con k4. No afirmes que menos k siempre mejora: puede bajar recall.

Ejercicio 2: el docente pide no inventar precios. Localiza `prompts/v3.txt`, refuerza la instrucción de solicitar una cotización cuando el contexto no incluya importes, guarda y repite. El prompt se lee por solicitud; cambios de config necesitan reinicio. Verifica también INT-COM: si la fuente no contiene precio, agregarlo al prompt como invento no resuelve el problema.

Ejercicio 3: llega una política nueva. Crea un Markdown breve de fuente interna simulada, añade su entrada al manifiesto con un id nuevo, versión y origen. Revisa y ejecuta `scripts.source_hashes`. Reinicia. Consulta una frase distintiva de ese documento y muestra id y versión. Para retirar la fuente anterior usa `active=false` y reinicia. No modifiques simultáneamente varios parámetros si quieres entender el efecto.

Ejercicio 4: el docente pide mayor prudencia. Cambia `min_score` en `config/rag.json`, por ejemplo de 0,06 a 0,12, reinicia y compara. Es un experimento, no un valor óptimo universal. Muestra tanto un caso conocido como uno fuera de dominio; explica el costo de abstenerse de más.

Ejercicio 5: cambia el SLA. Debes modificar `priority_for`, el documento INT-SLA y sus pruebas; después actualizar hash y ejecutar `pytest`. El sistema no interpreta una edición de SLA en texto para cambiar la función. Esta limitación es deliberada y debe decirse claramente.

Ejercicio 6: falla el proveedor. Detén Ollama, envía un ticket conocido y explica `provider_error`. No cambies a demo y lo presentes como recuperación transparente del LLM. Si necesitas continuar el recorrido, anuncia explícitamente el modo de práctica.

Banco de preguntas con respuestas ancladas al proyecto

1. ¿Por qué elegiste este caso? Porque triage exige clasificación, consulta documental y comunicación, tareas separables con evidencia observable. El escenario es ficticio; el impacto organizacional se plantea como hipótesis.
2. ¿Por qué RAG y no solo prompting? Un prompt general no contiene políticas ni guías concretas. RAG recupera fragmentos y permite mostrar procedencia. No garantiza verdad si la fuente está mal o la generación la interpreta mal.
3. ¿Por qué no fine-tuning? Las políticas pueden cambiar y necesito citas. Actualizar documentos es más sencillo en este corpus que entrenar un modelo. Fine-tuning podría servir para estilo o tareas repetidas, pero no sustituye conocimiento trazable.
4. ¿Qué fuentes son reales? Las páginas de Mozilla. Los archivos externos son síntesis propias con URL, fecha y transformación. Las políticas de Nexa TI y los tickets son simulados.
5. ¿Por qué 110 palabras y overlap 20? Es un punto de partida para procedimientos breves: conserva un bloque interpretable con redundancia limitada. Se puede evaluar otra segmentación; no es una constante óptima.

6. ¿Usas embeddings? Por defecto TF-IDF, que es vectorización léxica dispersa. Existe una ruta con embeddings densos de Ollama, pero debo diferenciar qué ruta ejecuté y evalué. No llamar semántico al baseline.
7. ¿Por qué no Chroma o FAISS? Nueve documentos caben en una matriz en memoria. Un índice externo no aporta suficiente beneficio en esta escala. Si aumentaran corpus y usuarios reevaluaría almacenamiento, índice y costos.
8. ¿Qué pasa si k aumenta? Puede mejorar cobertura y agregar ruido o redundancia. El presupuesto de contexto puede impedir incluir todos los candidatos. Lo mediría con precisión y recall, no solo viendo una respuesta.
9. ¿Qué pasa si ninguna fuente sirve? En lexical se aplica el umbral y, sin hits, se devuelve `insufficient_context`. Un puntaje alto tampoco prueba relevancia; por eso hay evaluación y revisión humana.
10. ¿Qué garantiza el JSON Schema? Estructura, tipos, campos y categorías válidas. No garantiza que la categoría o recomendación sea correcta. El validador de citas añade procedencia, pero falta comprobar respaldo semántico.
11. ¿Temperatura cero evita alucinaciones? No. Reduce aleatoriedad de muestreo, pero no hace verdadera una respuesta. Necesito fuentes, instrucciones, validación y revisión.
12. ¿Por qué prioridad por reglas? El alcance y la interrupción se declaran en campos estructurados. Una tabla simple da comportamiento repetible y evita usar la palabra urgente como única señal. El operador debe comprobar los datos ingresados.
13. ¿Qué diferencia hay entre precisión y fidelidad? Precisión evalúa documentos recuperados; fidelidad evalúa afirmaciones generadas contra esos documentos. Una respuesta puede ser fiel a un documento irrelevante.
14. ¿Por qué puedes tener 100 % en clasificación demo? Porque es un dataset sintético sencillo y un simulador por palabras clave. Esa cifra no representa un LLM ni predice generalización. Reporto el modo y necesito pruebas más variadas e independientes.
15. ¿Cómo evalúas alucinaciones? Ejecuto un LLM real, divido las respuestas en afirmaciones y los integrantes revisan cuáles están apoyadas en el contexto. Completo la ficha humana; no uso citas existentes como sinónimo de fidelidad.
16. ¿Cómo evitas prompt injection? Separo instrucciones y datos, restrinjo formato, no doy herramientas de ejecución y valido referencias. Son capas parciales: una instrucción hostil puede afectar contenido y debe probarse con inferencia real. No afirmo inmunidad.
17. ¿Qué ocurre ante conflicto de fuentes? La política interna manda en decisiones organizacionales y el prompt lo indica. No hay un detector semántico automático de conflicto: el operador revisa. Para producción añadiría resolución explícita y evaluación adversarial.
18. ¿Qué guardas en la traza? Ids, modo, modelo, hashes, configuración, tiempos y categoría; no el texto del ticket. El resultado descargado es diferente y puede contener información redactada del caso.

19. ¿Cómo sabes que la fuente no cambió? Comparo hash al cargar. El hash solo verifica bytes, no autoridad ni verdad. Las revisiones deben actualizar origen, versión y fecha con criterio editorial.

20. ¿Qué falta para producción? Organización validada, datos representativos, autenticación, permisos, almacenamiento, límites de concurrencia, observabilidad, política de privacidad y pruebas reales. El prototipo local no debe exponerse como servicio empresarial listo.

Cómo responder cuando no sabes

Resultados del prompt: v2 logró macro-F1 0,78; v3 logró 1,00 en la regresión de 20 casos sintéticos. Explica que se corrigió la taxonomía tras observar errores y que repetir esos casos no demuestra generalización. En desarrollo, v3 tuvo un error y macro-F1 0,905. Los 20 casos de regresión incluyen 18 generaciones y 2 abstenciones sin inferencia. No digas «el modelo nunca se equivoca» ni «todas las respuestas son fieles».

Ejemplo con evidencia disponible: en desarrollo, k=2 obtuvo precisión 0,929 y recall 0,952; k=4 obtuvo precisión 0,893 y recall 1,000. Puedes mostrar reports/COMPARACION_K.md y explicar que aquí bajar k quitó ruido, pero también evidencia. No confundas esta comparación del retriever con una mejora del prompt o de la generación.

Di qué sí puedes demostrar y qué está pendiente. Por ejemplo: «No medí embeddings reales en este informe; aquí evalué TF-IDF. Para compararlos mantendría dataset y prompt, registraría modelo y cambiaría únicamente el retriever». Una respuesta limitada y precisa es defendible; inventar una cifra o una integración no lo es.

Checklist de dominio personal

Puedo dibujar el flujo sin mirar. Puedo localizar el prompt. Puedo agregar una fuente y reconstruir el índice. Puedo distinguir demo y LLM. Puedo explicar un fallo de recuperación frente a uno de generación. Puedo interpretar precision/recall y macro-F1. Puedo modificar k o una regla y ejecutar pruebas. Puedo reconocer lo que no fue medido. Si alguna respuesta es no, practicar ese punto antes de la defensa.